

Deep Learning

Neural Networks Part I

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Today

- Neural Networks are computational models that are powering modern technologies such as ChatGPT, Google Translate and Alexa
- Inspirations from Neurobiology
- Artificial Neural Networks
- How to train Neural Networks
 - ▶ Backpropagation algorithm

Table of Contents

1. Analogy to Biological Neurons

2. Artificial Neural Networks

2.1 Perceptron

2.2 Multilayer Perceptron

3. Training Neural Networks

3.1 Backpropagation

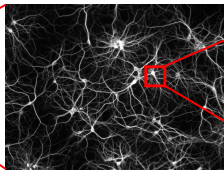
How Do We Recognize What We See?

- Every moment, our brain receives a flood of **visual information** – colors, shapes, textures, motion.
- From this, we instantly recognize:
 - ▶ a friend's face in a crowd
 - ▶ a stop sign at an intersection
 - ▶ a handwritten note on paper
- This seems effortless to us humans, but behind it lies **complex processing**:
 - ▶ Detecting patterns
 - ▶ Comparing with past experiences
 - ▶ Making fast decisions under uncertainty
- Visual recognition is **hard for machines**, but it's something humans do naturally – let's explore how we can model this.

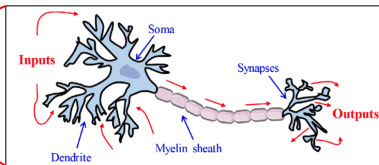
The Brain's Visual Processing



Human brain



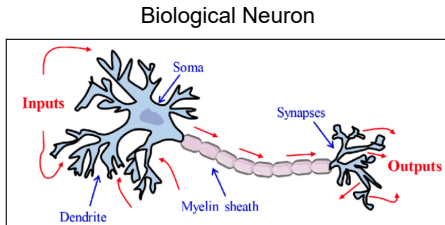
Network of neurons
(billions of them with trillions
of connections)



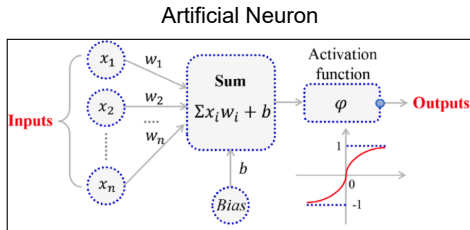
A single neuron

- **Neurons** are the basic building blocks of the human brain. It does the following:
 - ▶ Signals (electrical impulses) arrive at **dendrites**.
 - ▶ Cell body (or **soma**) integrates the incoming signals. If combined input exceeds a threshold, the neuron “fires”.
 - ▶ Signal travels down the axon to the next neuron through **synapses**.
- **Networks of neurons** allow the brain to process sensory input, recognize patterns, and make decisions.
- Q: Can we create a **computational model** that mimics how the brain works?

Biological Neuron versus Artificial Neuron



VS



Biological Neuron	Artificial Neuron
Dendrites receive input signals	Input nodes receive numeric values
Cell body integrates signals	Weighted sum of inputs is computed
Threshold triggers firing	Activation function decides output
Axon sends output to next cells	Output value passed to next layer

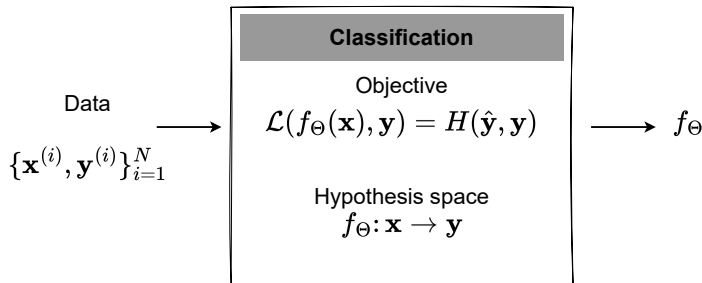
Artificial Neural Networks

- Definition on Wikipedia:
*"In machine learning, a neural network (also artificial neural network or neural net, abbreviated ANN or NN) is a computational model **inspired** by the structure and functions of biological neural networks."*
- **Inspiration:** We don't copy the brain exactly, but we borrow the core idea:
simple units + many connections = powerful computation.

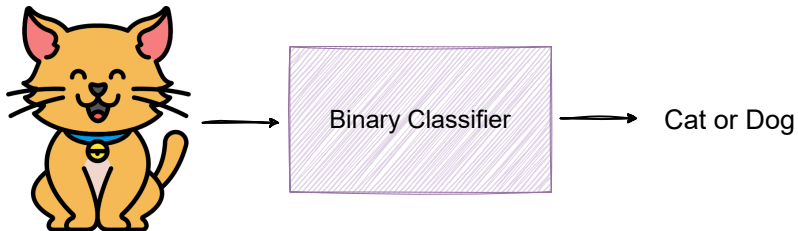
⁰<https://fastdatascience.com/ai-in-research/how-similar-are-neural-networks-to-our-brains/>

Recap slide from Lecture 4: Classification Problem

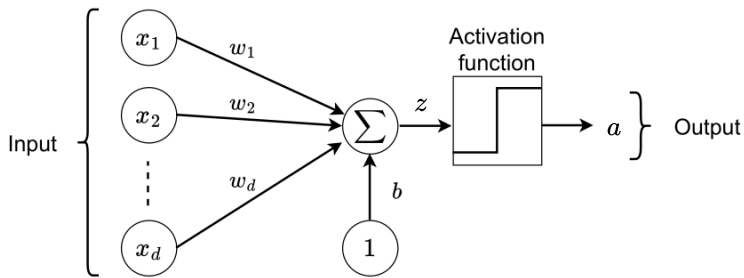
- We did not discuss a couple of things: hypothesis space and optimizer. What about them?
 - ▶ Hypothesis space will be discussed in the Neural Network lectures
 - ▶ Optimizer remains the same as linear regression problem



Binary Classification



Perceptron – A Simple Neuron

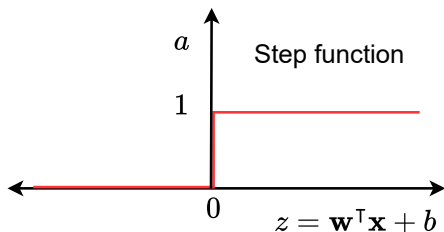


$$a = \begin{cases} 0 & \text{if } \mathbf{w}^T \mathbf{x} + b \leq 0 \\ 1 & \text{if } \mathbf{w}^T \mathbf{x} + b > 0 \end{cases}$$

Activation Function of a Perceptron

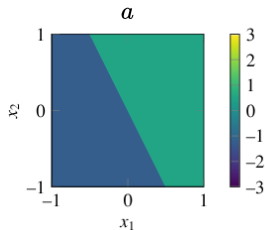
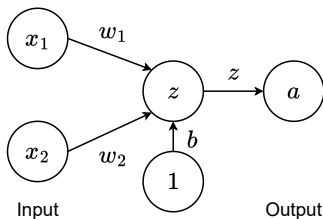
Let: $z = \mathbf{w}^T \mathbf{x} + b$

$$\text{Activation: } a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$



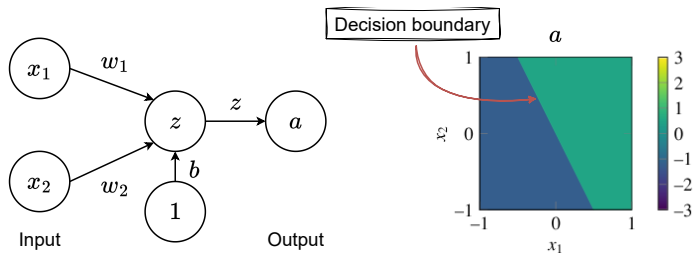
- An **activation function** is a mathematical rule that decides whether a neuron's output should "fire" (activate) or not, based on its input.
- For the perceptron, the activation function is a **step function**.

Perceptron as a Classifier



- Let's consider a simple perceptron with two inputs x_1 and x_2 and one output a .
- Let's assume that the weights have values: $w_1 = 2$ and $w_2 = 1$, and $b = 0$.
- Since a can take on values 0 or 1, we can think of this as a **classifier** that assigns a class label of 1 to the upper green part, and 0 to the lower blue part.

Perceptron as a Linear Classifier



- The decision boundary is given by:

$$\begin{aligned} z &= \mathbf{w}^T \mathbf{x} + b = 0 \\ &= w_1 x_1 + w_2 x_2 + b = 0, \end{aligned}$$

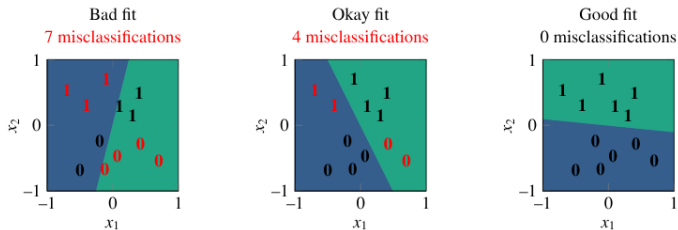
which is a linear function (also called as [linear hyperplane](#)).

Learning a Perceptron

- Just like other models (eg, linear regression, polynomial regression), we also need to "learn" the perceptron.
- Learning means adjusting the weights $\mathbf{w}$ and the bias b in order to minimize an objective function or a classification loss $\mathcal{L}$ (eg, cross entropy loss):

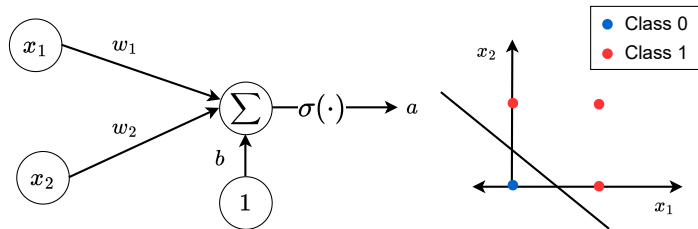
$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathbf{w}^T \mathbf{x}^{(i)} + b, y^{(i)})$$

- Adjusting $\mathbf{w}$ and the bias b results in shifting and rotating the decision boundary until you find a line that separates the data labelled as 1 from data labelled as 0.



What can a Perceptron do?

- Perceptron learns a **linear classifier**. Let's look at an example.
- Consider that we have four training data points, and the inputs are binary (i.e., $x_i \in \{0, 1\}$). The labels are also binary – $y \in \{0, 1\}$.



- Q: What are the values of the parameters, i.e., w_1, w_2, b , for such a classification problem?

What can a Perceptron do?

- Let's choose $w_1 = 1, w_2 = 1, b = 1/2$ and verify.
- Let's compute the activation a .
- To recall, $a = f(\mathbf{x}) = \begin{cases} 0 & \text{if } w_1x_1 + w_2x_2 + b \leq 0 \\ 1 & \text{if } w_1x_1 + w_2x_2 + b > 0 \end{cases}$

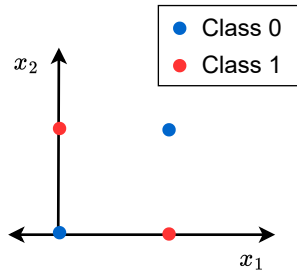
x_1	x_2	Activation (a)	y
0	0	$1 * 0 + 1 * 0 - 1/2 < 0$	0
1	0	$1 * 1 + 1 * 0 - 1/2 > 0$	1
0	1	$1 * 0 + 1 * 1 - 1/2 > 0$	1
1	1	$1 * 1 + 1 * 1 - 1/2 > 0$	1

- If you look closely at the table, this is the OR logic gate problem.
- A perceptron can also solve the AND and NOT logic gate problems. Verify.

What can a Perceptron do?

- Q: Can a perceptron linearly separate the the XOR logic gate problem?

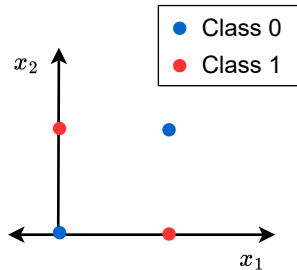
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



What can a Perceptron do?

- Q: Can a perceptron linearly separate the the XOR logic gate problem?

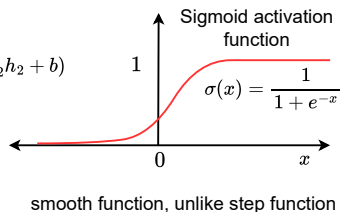
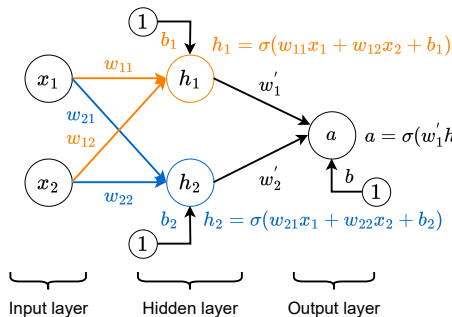
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



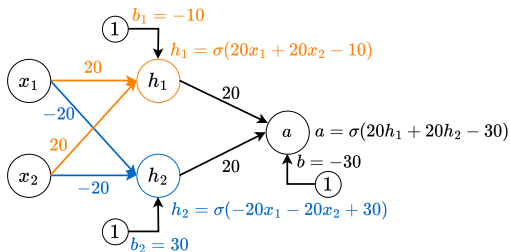
- A: The XOR operator can not be separated by a perceptron (it was proved by Minsky & Papert, 1969) because its not linearly separable.

Multilayer Perceptron – A Network of Neurons

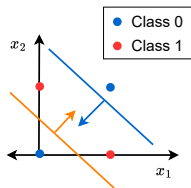
- We can solve the XOR problem, which is not linearly separable, using a **Multilayer Perceptron** (MLP) or a **Neural Network**. It was invented by Frank Rosenblatt in 1958.
- MLP contains the following:
 - ▶ Input layer
 - ▶ One or more hidden layers with non-linear activation functions
 - ▶ Output layer



Solving XOR with a MLP



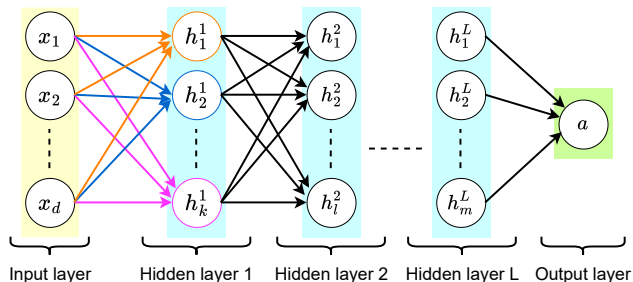
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0



$h_1 = \sigma(20 * 0 + 20 * 0 - 10) \approx 0$, $h_2 = \sigma(-20 * 0 - 20 * 0 + 30) \approx 1$, $a = \sigma(20 * 0 + 20 * 1 - 30) \approx 0$
 $h_1 = \sigma(20 * 1 + 20 * 0 - 10) \approx 1$, $h_2 = \sigma(-20 * 1 - 20 * 0 + 30) \approx 1$, $a = \sigma(20 * 1 + 20 * 1 - 30) \approx 1$
 $h_1 = \sigma(20 * 0 + 20 * 1 - 10) \approx 1$, $h_2 = \sigma(-20 * 0 - 20 * 1 + 30) \approx 1$, $a = \sigma(20 * 1 + 20 * 1 - 30) \approx 1$
 $h_1 = \sigma(20 * 1 + 20 * 1 - 10) \approx 1$, $h_2 = \sigma(-20 * 1 - 20 * 1 + 30) \approx 0$, $a = \sigma(20 * 1 + 20 * 0 - 30) \approx 0$

- The perceptron h_1 is responsible for one hyperplane, colored in **orange**.
- The perceptron h_2 is responsible for the other hyperplane, colored in **blue**.
- The output perceptron a is simply an **intersection** of the two above. This is a significant result because we can **combine several linear hyperplanes** to create a (very) **non-linear** decision boundary.

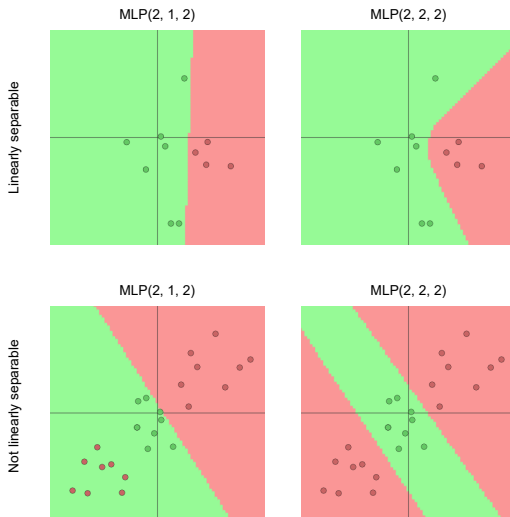
Going Deeper and Wider



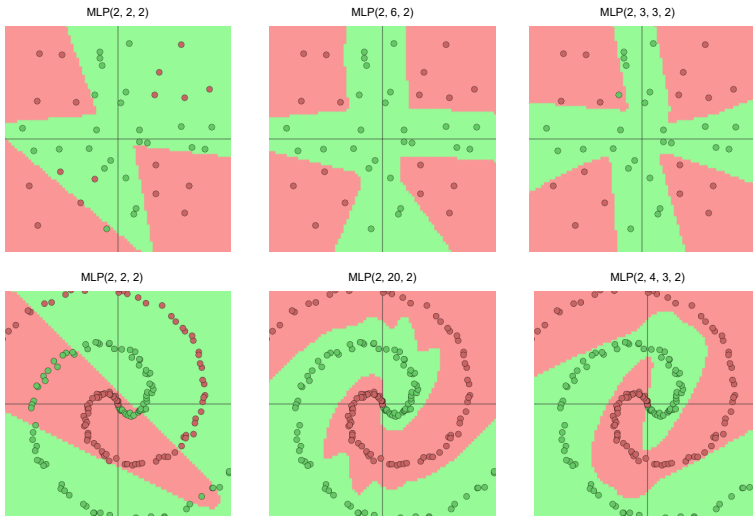
- We are not limited to one hidden layer and two perceptrons (also called as neurons) in a hidden layer. We can have an arbitrary number of hidden layers (a.k.a. **depth**) and an arbitrary number of neurons in each hidden layer (a.k.a. **width**).
- The deeper and wider we go, the more complex the decision boundary becomes.
- It was shown by (Cybenko, 1989) that a MLP (or NN) with just 1 hidden layer containing enough neurons and non-linear activation function can approximate any continuous function. This is called the **Universal Approximation Theorem**.

MLP for Toy Dataset Classification

- $\text{MLP}(2,1,2)$ means 2 inputs, 1 hidden layer with 1 neuron and 2 outputs.
- $\text{MLP}(2,2,2)$ means 2 inputs, 1 hidden layer with 2 neurons and 2 outputs.
- Notice that when there is 1 neuron in the hidden layer, the classifier is linear.
- Only when there are two neurons in the hidden layer, it can learn non-linear decision boundary. [Link to demo.](#)



MLP for Toy Dataset Classification



The First AI Winter (1960-1980s)

- In the 1963 Rosenblatt came up with the idea of adding more layers (a.k.a. going deeper) to make the NN very powerful. This is also why its also called **Deep Learning**.
- But the enthusiasm was short-lived because of several reasons:
 1. **Lack of hardware:** The cascade of transformations involve matrix multiplications, which were very expensive back then.
 2. **Theoretical reason:** It was theoretically proven that a NN with single layer and enough neurons can approximate any function. No reason to go deeper.
 3. **Technical reason:** There was no efficient algorithm to train the parameters of the hidden layers.
- Since it did not deliver on the promises, it led to research budget cuts and we entered into what we later called "The First AI Winter".

Deep Neural Networks – A New Hope

- In 1986, Rumelhart, Hinton, and Williams published a landmark paper showing that the [backpropagation algorithm](#) could efficiently train Deep Neural Networks (DNN), overcoming the main limitation that had blocked progress since the 1960s.
- The backpropagation algorithm is the workhorse in DNNs that allows to learn useful internal representations by adjusting hidden layer parameters.
- Computation power increased in the last 20 years – we could go deeper.



Neural Network

Recap of Optimization – Gradient Descent

- The cost function defined for linear regression model:

$$J(\Theta) = \frac{1}{N} \sum_{i=1}^N (\underbrace{\Theta^T \mathbf{x}^{(i)}}_{\text{model prediction}} - y^{(i)})^2$$

- The goal was to minimize the objective function so that we find the optimal parameters Θ^* .
- Similarly, for neural networks, we are interested in finding the values of $\mathbf{w}$ and b that minimizes the cost or the objective function:

$$\underset{(\mathbf{w}, b)}{\text{minimize}} J(\mathbf{w}, b)$$

$$\underset{(\mathbf{w}, b)}{\text{minimize}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\underbrace{f_{\Theta}(\mathbf{x})}_{\text{neural network prediction}}, y^{(i)})$$

Recap of Optimization – Gradient Descent

- Gradient descent is an iterative optimization algorithm that allows us to find the values of $\mathbf{w}$ and b .
- Working principle: Initialize the weights to something reasonable and repeatedly adjust them in the direction of the steepest descent.
- The weight update rule for a given parameter w_j is given as:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

- Intuitively, the term $\frac{\partial J}{\partial w_j}$ tells if we change w_j by a small value, how much J changes. Thus, we want to adjust w_j in a way that leads to a lower loss value.
- For all the parameters of the model we compute the **gradient** as:

$$\nabla_{\mathbf{w}} J = \frac{\partial J}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_D} \end{pmatrix}, \quad \frac{\partial J}{\partial b}$$

Backpropagation

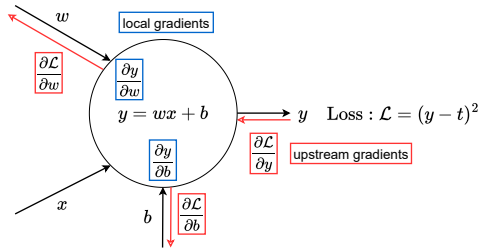


Backpropagation

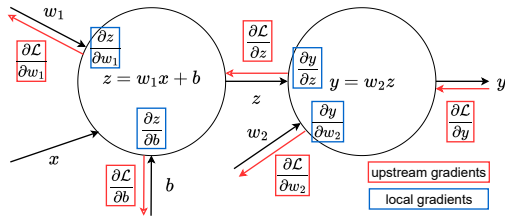
- The backpropagation algorithm use **chain rule** to compute the gradients in an efficient manner. We will see what “efficient” means.
- Let's consider we have the simplest regression problem, where we have one input feature x , and two parameters: w and b .
- For gradient descent we need: $\frac{\partial \mathcal{L}}{\partial w}$ and $\frac{\partial \mathcal{L}}{\partial b}$.
- Using chain rule:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial w} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial}{\partial w}(wx + b) \\ &= \frac{\partial \mathcal{L}}{\partial y} \cdot x\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b} &= \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial b} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial}{\partial b}(wx + b) \\ &= \frac{\partial \mathcal{L}}{\partial y} \cdot 1\end{aligned}$$



Backpropagation



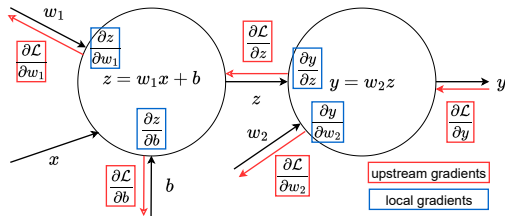
$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial}{\partial w_2}(w_2 z) = \frac{\partial \mathcal{L}}{\partial y} \cdot z$$

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial}{\partial z}(w_2 z) = \frac{\partial \mathcal{L}}{\partial y} \cdot w_2$$

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w_1} = \left(\frac{\partial \mathcal{L}}{\partial y} \cdot w_2 \right) \cdot \frac{\partial}{\partial w_1}(w_1 x + b) = \frac{\partial \mathcal{L}}{\partial y} \cdot w_2 \cdot x$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial b} = \left(\frac{\partial \mathcal{L}}{\partial y} \cdot w_2 \right) \cdot \frac{\partial}{\partial b}(w_1 x + b) = \frac{\partial \mathcal{L}}{\partial y} \cdot w_2 \cdot 1$$

Backpropagation



- We start at the back (or output layer) of the network, compute the gradient at each layer, and proceed toward the front (or input layer) of the network. This is how it got its name “backpropagation” (or backprop in short).
- The calculations suggest that even if a parameter (e.g., w_1) sits deep inside a network, to compute $\frac{\partial \mathcal{L}}{\partial w_1}$ we simply need to multiply the local gradient $\frac{\partial z}{\partial w_1}$ with the upstream gradient $\frac{\partial \mathcal{L}}{\partial z}$ (which is available from the previous round).
- In summary, backpropagation is an elegant way of computing gradients by **reusing previously computed gradients**.

Backpropagation

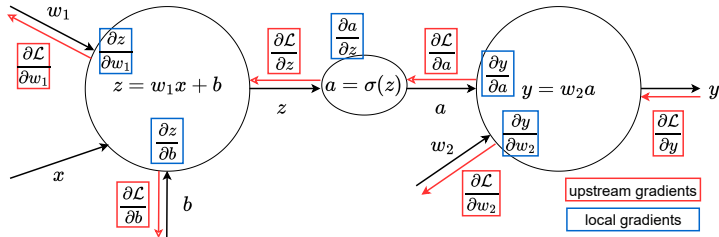


- Backpropagation is like playing the game of “telephone” .
- The formula for the computation of the gradient for a given parameter as:

$$\text{gradient} = \text{local gradient} \times \text{upstream gradient}$$

- The gradient then becomes the upstream gradient for the parameters of the previous layer of the network.

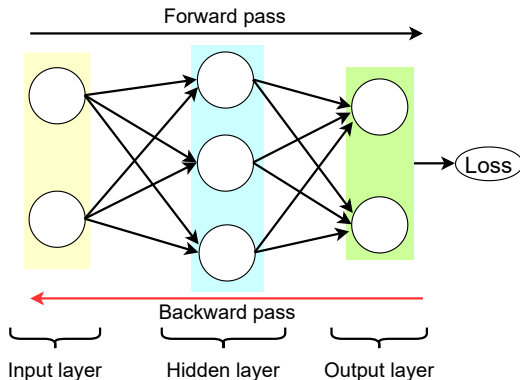
Backpropagation



- Q: Isn't there an **activation function** in the hidden layer of a neural network and won't it have an impact on the gradient computations?
- A: Yes $\times 2$. We will also need to compute the derivative of the activation function, i.e., $\frac{\partial a}{\partial z}$. The computations are slightly more involved¹.
- This is the reason why we want differentiable activation function; otherwise there will be no gradient flowing through the network. No gradient = no weight update!

¹Backprop with activation function: <http://neuralnetworksanddeeplearning.com/chap2.html>

Training a Neural Network



- At a very high-level, training a neural network involves the following steps:
 - ▶ **Forward pass:** Compute the network's output and calculate the error (loss).
 - ▶ **Backward pass:** Use the chain rule to propagate the error backward through the layers. Internally, it computes gradients of the loss with respect to each weight.
 - ▶ **Weight update:** Adjust weights using gradient descent (or variants like SGD).

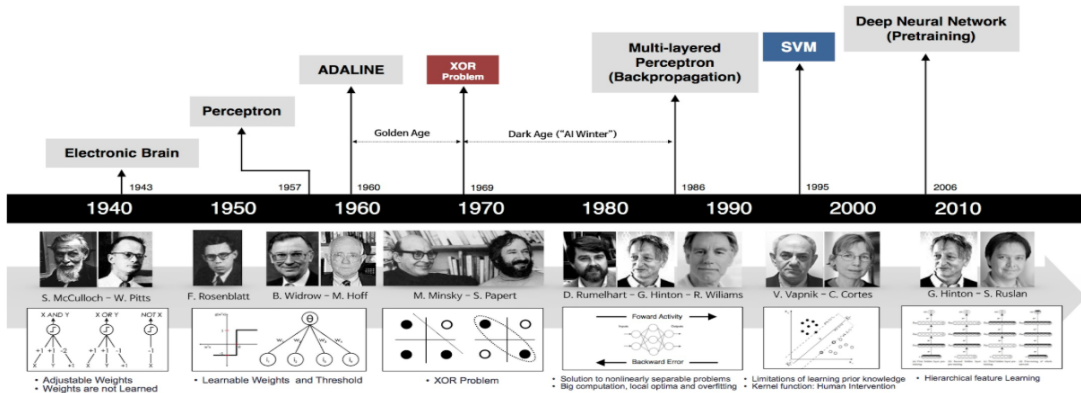
Neural Network Training: Pseudocode

```
1 import torch as nn
2 from torch.optim import SGD
3 # define the neural network
4 class Classifier(nn.Module):
5     def __init__(self):
6         super().__init__()
7
8         self.layer_hidden = nn.Linear(2, 10)
9         self.relu = nn.ReLU()
10        self.layer_out = nn.Linear(10, 1)
11
12    def forward(self, x):
13        x = self.layer_hidden(x)
14        x = self.relu(x)
15        x = self.layer_out(x)
16        return x
17
18 model = Classifier() # the neural network
19 y_pred = model(data) # forward pass
20 loss_fn = nn.MSELoss()
21 loss = loss_fn(y_pred, y) # compute loss
22 loss.backward() # backpropagation
23 optimizer =SGD(model.parameters(), lr=learning_rate)
24 optimizer.step() # do gradient descent
```

The Second AI Winter (1990s-2005)

- Despite the encouraging innovations at the end of 1980s, the success was short-lived.
- Kernel Machines (e.g. SVMs) suddenly become very popular.
 - ▶ Accuracies similar to those of neural networks in the same tasks.
 - ▶ Much fewer heuristics and parameters.
 - ▶ Nice proofs on generalization.
- Computation power was still a bottleneck, deep NN could not be trained, yet.
- Large annotated datasets were hard to come by.

History of Neural Networks



Next Lecture

- Neural Networks is a very large field of study and evolving faster than you can say the words “Neural Networks”.
- It is nearly impossible to cover everything in a few lectures! We will try to get the most important bits.
- Things we are going to cover in the next lecture:
 - ▶ Different design choices in NN.
 - ▶ Different kinds of NN architecture.
 - ▶ Applications of NN.

References and Links

- For additional reading materials refer to the following links
 - ▶ [“CS 229 Lecture Notes”](#)². Chapter: 7
 - ▶ [“CS231n Lecture Notes”](#)³.

²https://cs229.stanford.edu/main_notes.pdf

³<https://cs231n.github.io/>

Credits and Acknowledgements

- Some slides have been adapted from:
 - ▶ Torralba, Isola and Freeman
 - ▶ W. Killian
 - ▶ S. Yeung
 - ▶ V. Lavrenko